Kubernetes as a Virtual Machine Management System

A Comparative Study of KubeVirt and Traditional Approaches

EMIL KARLSSON

Master in Computer Science
Date: February 22, 2024
Supervisor: Han Fu
Examiner: Cyrille Artho
School of Electrical Engineering and Computer Science
Host company: Net Insght AB
Swedish title: Swedish title TBD

Abstract

Sammanfattning

Contents

1 Introduction **2**
1.1 Background 2
1.2 Research Question 2
1.3 Problem 3
1.4 Purpose 3
1.5 Goals 3
1.6 Research Method 3
1.7 Limitations 3
1.8 Structure of the thesis 3

2 Background **4**
2.1 Virtualization Fundamentals 4
2.2 Understanding Hypervisors 6
2.3 VM Management Essentials 8
2.3.1 Comparison of VM Management Systems 9
2.4 Containers and Kubernetes: A Paradigm Shift 13
2.4.1 Containers Explained 13
2.4.2 Orchestration Using Kubernetes 14
2.5 Containers vs. Virtual Machines 15
2.5.1 Comparative Overview 16
2.5.2 Decision Factors 18
2.6 Integrating VMs into Kubernetes with KubeVirt 18
2.6.1 KubeVirt Overview 18
2.6.2 Features and Capabilities 18
2.7 Evaluating VM Management Systems: Kubernetes with KubeVirt 18
2.7.1 Strategies for Evaluation 18
2.7.2 Metrics for Comparison 18
2.8 Related Work 19

3 Methods **20**

4 Results **21**

5 Discussion **22**

6 Conclusions **23**

Bibliography **24**

A Something Extra **29**

;

Chapter 1

Introduction

1.1 Background

1.2 Research Question

1. What performance, scalability, and operational efficiencies can be gained by integrating virtual machines in Kubernetes?

   This could stem from using a single resource scheduler. Several metrics will be used to evaluate this, as described in the background and related work.

2. What are the current challenges and limitations in managing virtual machines and containerized applications in separate environments?

   e.g. running CloudStack/OpenNebula/OpenStack alongside Kubernetes.

3. How can system administration be simplified by running virtual machines in Kubernetes using KubeVirt?

   e.g. if there is any efficiencies to be gained in terms of usability, for instance:

   - After clicking *Deploy VM*, how quickly can the virtual machine be accessed?
   - How many steps are there before I can access my virtual machine?
   - Are there any perceived benefits when interacting with the system, compared to the previous one?

1.3 Problem

1.4 Purpose

1.5 Goals

1.6 Research Method

1.7 Limitations

1. This project will not investigate any solutions for shortcomings in KubeVirt itself, for example, finding alternatives for features that exist in existing virtual machine management systems but not in KubeVirt. As such, this project acknowledges that KubeVirt might not be a complete replacement to traditional virtual machine management tools (also described in [1]), as it might not offer all the features of an IaaS system. Therefore, only features that exists in KubeVirt and the compared system will be considered when testing. However, any lack of features (KubeVirt or not) will be brought up in the discussions.

2. From the description of this project above, there are also mainly two scenarios that will be evaluated.

   - A system uses traditional virtual management (OpenNebula, CloudStack etc.).

     In this scenario Kubernetes with KubeVirt is seen mainly as a 1:1 replacement.

   - A system uses both traditional management and Kubernetes.

     This scenario is more aimed towards unifying the environments using KubeVirt.

1.8 Structure of the thesis

Chapter 2

Background

In this chapter we will discuss how virtualization has evolved over time to reach the pointer where we are today.

2.1 Virtualization Fundamentals

Virtualization is a technology [2] that enables partitioning or grouping computing resources into operating environments using methods such as partitioning, emulation, and time-sharing. Virtualization could occur in various forms, such as network virtualization, storage virtualization, and server virtualization. More commonly used, though, is hypervisor based virtualization [3], which is a software layer that creates virtual operating environments, or more commonly referred to as virtual machines (VMs).

A VM is a emulation of a physical computer [4] and is perceived as *real* computer by the application running inside it even though underlying resources are, more often than not, shared with other VMs. For this reason, virtualization is often used to improve resource utilization and to isolate applications from each other. Such a setup improves flexibility of the computing environment, but it implies that the underlying mechanism is formally different from that of a physical machine. For instance, hardware abstraction is used to decouple the VM's perceived hardware capabilities from the physical hardware. Abstracting the underlying hardware allows for *virtual* computing resources, such as virtual CPUs and virtual memory, either by grouping or partitioning the physical resources.

Furthermore, the abstraction layer can exist in multiple different layers depending on the virtualization technology used, ranging from the hardware level to the application level, some of which are listed below:

- Full virtualization (FV): Provides a complete emulation of the underlying physical hardware and creates a complete virtual environment. FV enables fully decoupling the VM from the underlying resource, and does not require any modification [2][5][6].

- Paravirtualization (PV): A subset of FV aiming to improve the VM performance by reducing the amount of resource that are emulated. The VM is presented with an image that is similar, but not identical, to the underlying hardware. As such, the guest OS is aware it is running inside a VM [5][6].

- Application virtualization (AV): A method where an application is encapsulated in a virtualized lightweight environment, such it perceived it is being run on a specific platform. Application virtualized using AV act as a portable and self-sufficient unit, thus often used to package and distribute applications, such as the JVM [2][7].

Define virtualization and discuss its significance in modern IT infrastructures, including the evolution from physical to virtual resources.

Benefits and Challenges

Many of benefits with virtualization stems from the fact that there exists an abstraction layer above the physical hardware. The abstraction layer decouples software from hardware, e.g. vendors with different I/O devices can be used in the same environment, allowing for a more flexible and scalable computing environment as whole where hardware is turned into simple pool of computing resources [4]. As such, there are numerous benefits with virtualization, a few of which are listed below:

- Resource utilization: Virtualization allows for better utilization of hardware resources, as multiple VMs can run on the same physical hardware. This is achieved by sharing the underlying resources, such as CPU and memory, between the VMs [2].

- Isolation: VMs are able to isolate their entire state from one another, including operating system (OS), filesystem and application stack. Isolation is also beneficial in scenarios where reproducibility is key, such as in testing and development environments [8].

- Flexibility: VMs not tied to a specific hardware, and can be moved between different physical resources.

- Management: VMs can be created and destroyed on demand, and can be managed using software tools [2].
- Virtual hardware: A VM can be configured with hardware that does not exist in the physical environment, such as a virtual network card or a virtual hard drive [2].

However, the portability made possible by virtualization comes at a cost of performance. Since, at the core, a VM is a complete but emulated system, it means the every instructions issued must be interpreted in software. As such, despite what level of abstraction layer is used, there is an inescapable added overhead when using virtualization. A common hypervisor such as KVM (see 2.2) introduces an overhead of 5-10% [9], and the overhead can be even larger when using a full virtualization approach.

Outline the advantages virtualization offers, such as resource optimization and isolation, alongside potential challenges, such as complexity and overhead.

2.2 Understanding Hypervisors

A hypervisor is a software layer that enables managing multiple VMs on a single physical machine [3] by allowing multiple operating systems (termed as *guests*) to share the same hardware [10]. Hypervisors, often referred to as virtual machine monitors (VMMs), are responsible for creating and managing VMs, and for providing the necessary abstractions to the VMs.

Hypervisors can be categorized into two types, *type 1* and *Type 2*, based on their operational context and how they interact with the underlying hardware [10][11]. While the goals and purpose of type 1 and Type 2 hypervisors is the same, there are some core differences. type 1 hypervisors, commonly referred to as *bare-metal* hypervisors, run directly on top of the physical hardware and do not require an OS underneath. A type 1 hypervisor is therefore scalable to terabytes of RAM and hundreds of CPU cores [11]. However, without any other additional software, it is also itself responsible to schedule and allocate required system resources to the VMs. Type 2 hypervisors, also known as *hosted hypervisors*, relies on an underlying OS to manage the calls to the CPU and memory resources. The presence of an underlying OS inevitably introduces latency, which is why Type 2 hypervisors are often used for development and testing purposes [10].

Furthermore, hypervisors can be further categorized into *monolithic* and *microkernelized* hypervisors [12]. Monolithic hypervisors are built as a single

large software stack, managing all the VMs and their resources, and hosting drivings for all hardware access. Having all drivers reside within the hypervisor means that the such as hypervisor will only run on selected hardware. However, it also means no operating system is required at all, as the hypervisor acts as a operating system by supporting all the operations of its VMs. Microkernelized hypervisors, on the other hand, does not require devices drivers to be present in the hypervisor. Instead, the drivers are installed in a form of managing VM, often referred to as parent VM. Each VM created by such a hypervisor is thus referred to as a child VM, as it uses the parent VM's drivers to access the hardware. This approach is more flexible, as it can allow the hypervisor to run on any hardware that supports the parent VM's operating system.

In the realm of hypervisors, the concept of *security rings* is often used to describe the level of privilege different part of a hypervisor has. A security ring represents a hierarchical protection domain, where the innermost ring (ring 0), usually reserved for the OS kernel, has the highest privilege, and the outermost ring (ring 3), usually reserved for user applications, has the lowest privilege. The hypervisor itself is often placed in ring 0, its VM kernel space in ring 1, and the VM user space in ring 3. Security rings are thus used to define execution level to provide a system to limit execution capabilities of VMs in order to regulate access to shared memory. This enabled a strong isolation layer between the VMs, and between the VMs and the hypervisor, thus a key part of a hypervisor's security model [12][13].

Explain what hypervisors are, differentiating between type 1 (bare-metal) and Type 2 (hosted) hypervisors, and their operational contexts.

Key Hypervisors

There are several hypervisors available, each with its own set of features and capabilities, some of which are listed below:

- VMware ESXi: A lightweight type 1 monolithic hypervisor often used in enterprise environments. ESXi is known for its ultra lightweight footprint stretching merely a few hundred megabytes, and its ability to run directly on the hardware without an underlying OS [12].

- Microsoft Hyper-V: A type 1 microkernelized hypervisor that is part of the Windows Server operating system. As a microkernelized hypervisor, Hyper-V differentiates between parent and child VMs, and uses the parent VM to manage hardware access for the child VMs [12][14].

- Citrix Hypervisor: Formerly known as XenServer, is a type 1 microkernelized hypervisor based on the Xen Project. It runs using a modified Linux kernel in a VM, called Dom0, that manges hardware I/O and interacts with the other virtual machines, called DomU guests [6][12].
- KVM: A type 1 hypervisor that is part of the Linux kernel. As such, KVM uses most of the functionalities from the Linux kernel, such as the scheduler and memory management. KVM has become a popular alternatives to proprietary hypervisors, such as VMware ESXi and Microsoft Hyper-V, as it is open-source and free to use [12][15].

Delve into specifics of widely used hypervisors such as VMware's ESXi, Microsoft Hyper-V, and KVM, highlighting their features, performance aspects, and use cases.

2.3 VM Management Essentials

A management system for VMs involves a set of tools and processes that are used to create, deploy, and manage VMs [16]. The management system is responsible for managing the entire lifecycle of a VM, including provisioning, deployment, monitoring, and scaling. Not be confused with a hypervisor, a VM management system lives on a higher-level and aims to provide an abstraction layer in the form of an interface to allow interaction with the VMs, often enabling the support of a set of hypervisors [17]. As such, a VM management system unifies hypervisors, typically through an higher-level API, such as a CLI or a web-based user interface.

Concepts and Features

With a higher abstraction layer, a VM management system brings a set of concepts to simplify managing VMs, such as clustering, availability, and scalability [17]. Clustering includes grouping a set of resources, such as VMs or storage nodes, to provide a image that is perceived as a single resource. Availability refers to the ability to keep a resource active or running, which is, more often than not, clustered resources. Scalability involves technologies to ensure resources are able to manage an increase or decrease in computational load, such as increasing the amount of virtual cores in a VM if it's overloaded.

Infrastructure as a Service

A VM management system is often part of a larger cloud computing infrastructure, such as Infrastructure as a Service (IaaS). IaaS is a lower-level cloud computing model that aims to eliminate the need to install and manage physical hardware by providing virtualized computing resources, such as VMs, virtual network and storage.

Discuss essential VM management capabilities, including live migration, snapshot management, and scalability.

2.3.1 Comparison of VM Management Systems

Managing VMs can be done in many ways, and there are many VM management systems available, each with its own set of features and capabilities. Many such systems are a part of a larger platform, such as an IaaS or even as large as a cloud platform. Some of the most prevalent platforms that has been used in the industry are OpenStack, CloudStack, and OpenNebula, and some of the most popular public cloud platforms are Amazon EC2, Azure Virtual Machines, and Google Compute Engine.

OpenStack

An open-source IaaS platform that offers a large set of features to manage VMs, network, and storage [16]. OpenStack's architecture is inherently modular, and it is designed to be deployed in a distributed manner [18]. The architecture includes three major parts:

- *Nova*: The compute service that manages VMs and provides an interface to manage VMs. Nova support all operation regarding the life cycle of a VM, such as creating, deleting, and migrating VMs. Nova is itself divided into multiple copmonents, each employing a *shared-nothing* and *messaging-based* policy, which means every component is independent and communicates with other components using a message queue.

- *Swift*: The object storage service that provides a scalable and redundant storage system for VMs. It is a long-term storage for static data, such as images and backups, and is comparable to Amazon S3.

- *Glance*: The image service that provides a catalog of VM images that can be used to create new VMs.

OpenStack supports features such as live migration, snapshot management, and scalability, and is often used in large-scale cloud environments [16]. However, these features are often hypervisor-specific, and requires both support from the hypervisor and specific configuration to work [19].

CloudStack

An open-source IaaS platform and cloud computing software that manages computing, networking and storage resources, and is used to build and manage public and private cloud environments [20]. On a high level, CloudStack is built in a monolithic fashion, encapsulating all the management features into two software packages called the management server and agent. The management server is responsible for managing the lifecycle of VMs, including provisioning, deployment, and scaling, and is comparable to a control node. The agent, however, is installed on each node in the cloud and is responsible for low-level management task of the VMs on that node, such as interacting with the underlying hypervisor. While it is a monolithic system, CloudStack is designed to be horizontally scalable, and can be deployed in a distributed manner with multiple management servers [20].

CloudStack supports a large variety of hypervisors, such as VMware, KVM, and XenServer, and storage services, such as Amazon S3, allowing it to function as an abridgment layer for compute, network, and storage to unify a wide range of existing infrastructure [20]. Features such as live migration, snapshot management, and scalability are supported, but are often hypervisor-specific and requires specific configuration to work [21].

OpenNebula

An open-source IaaS platform and toolkit used to provide open, scalable, and comprehensive management of virtualized data centers [16]. OpenStack employs a modular architecture divided in three layers [22][23]:

- *Tools layer*: Provides a set of interfaces such as a CLI and web-based user interface to interact with the system, but also high-level management tools such as scheduling.
- *Core layer*: Provides the core functionalities of the system, such as host management, VM management, and virtual network management.
- *Driver layer*: Provides an abstraction on top of the underlying infrastructure to enable the system to support a wide range of hardware, including networking, storage, and hypervisors. As such, the driver layer

is responsible for allocation, deallocation and monitoring of resources, such as VMs and storage.

By dividing the system into these layers, OpenNebula provides modularity that enables the system to be extended and customized to fit specific needs, such as integrating with other systems or using OpenNebula to build a private cloud.

Amazon EC2

Amazon Elastic Compute Cloud (EC2) is a web service that provides on-demand compute resources over the cloud [24][25]. EC2 is part of Amazon Web Services (AWS), and is designed be scalable and flexible, and is often used to host general-purpose applications, including web applications, databases and enterprise applications. While EC2 platform uses Xen-based hypervisors (see 2.2) under the hood [26], it acts as a complete abstraction layer over the underlying hardware and hypervisors, and instead provides a rich set of tools to manage the VMs.

EC2 provides a large set of preconfigured templates called Amazon Machine Images (AMIs) that can be used to create new VMs, as well as a configurations in terms of specifications. This enables EC2 to provide a wide range of VMs, such as general-purpose, compute-optimized, memory-optimized, and storage-optimized VMs [24]. Features such as live migration, snapshot management, and auto-scaling are supported,

Azure Virtual Machines

Azure Virtual Machines is a service provided by Azure to manage VMs in the cloud [27], and is designed to be a cost-effective alternative to on-premises hardware [28]. Azure Virtual Machine provides a set of preconfigured VMs that can be used to create VMs for specific purposes, such as general-purpose or high-performance computing (HPC).

Behind the scenes Azure Virtual Machines runs on Hyper-V (see 2.2) and provides a set of features such as live migration, snapshot management, and auto-scaling [29].

Google Compute Engine

Google Compute Engine is a service provided by Google Cloud to manage computing resources in the cloud [30]. Compute Engine consists of three major parts: VMs, storage, and networking, where VMs is the core component.

VMs are offered in a wide range of configurations, such as standard, high-memory, and high-CPU, making them suitable for different computational workloads [31].

Compute Engine uses KVM as its hypervisor (see 2.2), and provides many features such as live migration, snapshot management, and auto-scaling [30][26].

Comparison

OpenStack, OpenNebula, and CloudStack are all examples of software that can be used to build private cloud platforms [32][33]. A private cloud is an on-premise cloud that hosted within an organization, more often than not for that organization's internal use. A common denominator for these systems is that they are open-source, and that they are highly customizable and extendable. As such, much of the configuration and management of the system is done by the user, and the system can be integrated with other systems to fit specific needs. This means many of the conveniences, such as preconfigured VMs, and features, such as live migration, often requires specific configuration and setup.

On the other hand, VM management systems part of a public clouds, such as Amazon EC2, Azure Virtual Machines and Google Compute Engine, are designed to be more user-friendly and to provide a more complete abstraction layer over the underlying hardware and hypervisors [32]. As such, many of the conveniences and features are provided out of the box, and are perceived as hypervisor- and hardware-agnostic. This means that the user does not need to worry about the underlying hardware and hypervisors, and can instead focus on managing the VMs.

Both private and public cloud solutions for VM management are designed to be scalable and flexible, and support a wide range of use cases (although private cloud solutions scales worse in practice [33]). Which solution to use depends on the specific needs of the organization, and the level of control, customization needs, and the required security level of the system.

Briefly introduce traditional VM management systems like OpenStack, CloudStack, and OpenNebula, focusing on how they implement these core features.

2.4 Containers and Kubernetes: A Paradigm Shift

Besides the common cloud service models (IaaS, PaaS, and SaaS), another service models has emerged, called *Container as a Service (CaaS)* [34]. CaaS is a container based cloud service model that has gained significant traction due to the lightweight and portable nature of a container, allowing for simple application bundling that is able to run in a vast range of environments.

Containers are often used in conjunction with a container orchestration platform, such as Kubernetes, to manage the lifecycle of the containers, including deployment, scaling, and monitoring. As such, they are often used to build, test, and deploy applications in a consistent manner. ThiA common use case for containers is to run microservices, which is a software architecture that employs a set of small, independent and loosely coupled applications [35].

2.4.1 Containers Explained

Containers are a form of virtualization that offers a horizontally scalable and lightweight alternative to VMs [36][37]. Unlike VMs, containers do not require a hypervisor to run, and instead run on top of an OS and shares the OS kernel with other containers. By not requiring a hypervisor, containers are able to run with less overhead than VMs, making their lifecycle relatively fast and efficient compared to VMs.

The Open Container Initiative (OCI) is a standardization effort that was created to ensure that containers are portable and interoperable across different platforms [38]. The OCI standard defines a runtime specification, image specification, and distribution specification, allowing for a container to be consistent in terms of its runtime environment, image format, and distribution method.

Creating, deploying, and managing containers is using a container runtime, such as containerd [39]. Most runtimes, including containerd, implement the OCI runtime specification, and are able to run containers in a consistent manner. This also suggests that the OCI can be viewed as an abstraction layers, where, for instance, the OCI runtime specification acts as an intermediary between the container runtime and the OS kernel.

Define containers, their architecture, and how they differ from VMs in terms of overhead and isolation.

2.4.2 Orchestration Using Kubernetes

Kubernetes is an open-source container orchestration platform that manages containers. It maintains a set of abstraction layers to manage the life cycle of containers and to provide a consistent environment for the containers to run in, such as scheduling, scaling, and networking [37].

Kubernetes is inherently declarative, meaning that instead of issuing commands to perform a specific task, such as starting a container, Kubernetes is given a desired state for a set of resources, and it is responsible for ensuring that the current state of the resources matches a desired state [37][40]. As such, Kubernetes provides auto-restart, re-schedule, and replication control to ensure a resource stays in the desired state. By specifying the desired state, an entire application is able to be contained in a single configuration file, including its container, networking, storage, and security requirements.

Some of the abstractions used in Kubernetes includes *Pods*, *Services*, and *Deployments* [40]. A Pod is the smallest deployable unit in Kubernetes, and represents a group of one or more containers that are scheduled to run on a node. A Service represents a single-point-of-entry communication layer for a set of Pods, and is used to provide a consistent way to access the Pods, such as load balancing. A Deployment is a higher-level abstraction on top of Pods that is used to manage the life cycle of the Pods, such as scaling and rolling updates. A Deployment does not represent an executable resource, but instead describes a desired state in which a set of Pods should be in. There are many other abstractions in Kubernetes, each with its own set of features and capabilities that together creates a feature complete platform for container orchestration.

A Custom Resource Definition (CRDs) is a form of extension for Kubernetes resources that allows for the creation of custom resources [40][41]. CRDs are used to extend the Kubernetes existing abstractions to provide additional features and capabilities to allow for a more feature complete platform when custom behavior is needed. For instance, a CRD can be used to enforce certain fields to be specified in order to create the resource, or to provide a custom validation logic [42].

Kubernetes uses a client-server architecture, where the server, often referred to as the control plane, is responsible for managing the state of the cluster, and the client, often referred to as the worker node, is responsible for running the containers [40]. The control plane runs system-related services, such as the API server, scheduler, and controller manager, and is thus responsible for managing the state of the cluster. The API server can be viewed as the primary service for a Kubernetes cluster as it is the main management point for

the cluster, receiving and processing request from the client. The worker node, on the other hand, is responsible for running the containers containers. The worker node consist of two primary components: the kubelet and the kube-proxy. The kubelet is a service that includes a container runtime, which runs containers in according to requests from the control plane. Meanwhile, the kube-proxy service manages the network communication for the containers. To interact with Kubernetes form a user perspective, a client is used to send requests to the API server. The client can be a CLI, such as kubectl, a web-based user interface, or through a programmatic interface such as a REST API [40]. The CLI tool kubectl is the most common way to interact with Kubernetes, and is used to create, delete, and manage resources in the cluster.

Through its abstractions, life cycle management, and declarative client-server architecture, Kubernetes provides a consistent and feature rich platform for container orchestration.

Introduce Kubernetes, detailing its components, functionalities, and how it revolutionized container management.

2.5 Containers vs. Virtual Machines

Virtualization has enabled the commoditization of computing resources due to the benefits of being able to map virtual to physical hardware (see 2.1), and has been key in enabling cloud computing [37][43]. It has enabled the creation of flexible and scalable computing environments, and has been used to build and manage large-scale cloud platforms. Virtualization has mainly brought two technologies, hypervisors and containers. Hypervisors are used to create VMs, including all associated aspects such as an OS. Containers, on the other hand, are used to create a lightweight and portable environment for an application to run in, and are often run inside VMs. As such, a container comes with less overhead when compared to a hypervisor. However, by its nature, a VM is able to provide a more isolated environment than a container, leading to a more secure environment. Thus, the benefits of containers does not come without a cost, and there is a trade-off between the two technologies; one is lightweight, portable, and quick to start, while the other is more independent and secure.

2.5.1 Comparative Overview

While containers and hypervisors are both technologies brought by virtualization, they operate on distinct levels of abstraction [43]. Containers operate on the application level using OS-level virtualization, which leads to sharing parts the OS kernel with other containers. A hypervisor on the other hand is able to virtualize the entire hardware stack, including the entire OS, depending on the type of hypervisor used (see 2.2). As such, the two technologies have different use cases, and are often used in conjunction with each other.

Isolation

The main difference between containers and VMs in the level of isolation they provide. Containers share the OS kernel with other containers, and VMs do not. While this means a container is able to share resources more efficiently, it also means that a container is subject to a larger attack surface, as a vulnerability in the OS kernel can affect all containers running on the same host. VMs, on the other hand, are able to provide a more isolated environment, as they run on their own OS and are not subject to the same attack surface.

Another aspect of isolation is the how co-located containers and VMs are able to share resources. When sharing resources, such as CPU and memory, containers are vulnerable to the noisy neighbor problem [44]. The noise neighbor problem occurs when a resource sharing mechanism, such as a scheduler, is unable to provide a fair share of resources to a container. Some scenarios include when a container is able to consume all available resources, or when a container intentionally consumes all available resources to starve other containers. VMs, on the other hand, are able to provide a more fair share of resources, as they are able to provide a more isolated environment [43]. Hypervisors, on the other hand, are able to provide a more fair share of resources, as they are able to provide a more isolated environment, leading to more resilient and predictable performance. However, a VM's isolation does not mitigate the noisy neighbor problem for all resource types. For instance, a VM is still vulnerable to the noisy neighbor problem when sharing network related resources, such as network bandwidth and network-based storage.

Overcommitment

Overcommitment is a technique used to improve packing efficiency, often referred to as *consolidation ratio*, and it is often necessary to overcommit physical resources [43][45]. Containers use the overcommitment mechanisms built

into the OS itself, such as CPU overcommitment by multiplexing and memory overcommitment through virtual memory and memory swapping. Hypervisors, on the other hand, provide overcommitment by mapping virtual hardware devices onto the actual physical hardware, creating virtual computing resources that are not directly tied to the physical resources. Depending on the consolidation ratio, overcommitment for hypervisors can lead to a degradation in performance as the guest OS believes it has access to more resources than it actually has. Memory overcommitment, though, is a more complex issues in hypervisors than in containers due to the way hypervisors allocate memory. Unlike containers, hypervisors assign a static amount of memory to a VM when it is created. However, methods such as memory ballooning exist to address this limitation [46]. Memory ballooning is a technique used to treat memory as a shared resource across VMs, and allow the hypervisor to reclaim memory from VMs when it is needed elsewhere, thereby reducing the need for memory overcommitment.

Live Migration

Live migration is a technique used to move a running resource from one host to another without interrupting the services provided by the resource [47]. The technique is often used to balance the load of the hosting nodes or to perform maintenance on the hosting node.

Live migration of VMs is a mature and wildely used in large data centers, and works by copying memory pages from the source host to the destination host, then transferring the control of the VM from the source hypervisor to the destination hypervisor. As VM disks are often stored on a shared storage, such as a Storage Attached Network (SAN), the VM's disk is not moved during the live migration, and the VM can safely continue to run after its running state has been transferred to the destination host.

Live migration for containers, on the other hand, is not as mature as for VMs. Unlike VMs, containers require process migration, which is not as reliable. However, there are several projects that aim to provide live migration for containers, such as Checkpoint-restore in Userspace (CRUI) [48][49]. CRUI operates by checkpointing the state of a container, such as its memory and file descriptors, and then restoring the state on another host. Another approach for the live migration of containers involves simply terminating them and restarting them on another host. This approach is often sufficient in practice, as containers are typically stateless.

Elaborate on the distinct characteristics of containers and VMs, including

performance, isolation, and use cases.

2.5.2 Decision Factors

The choice between containers and VMs depends on the specific use case, and there are several factors to consider when deciding which technology to use.

Discuss factors influencing whether to deploy applications in containers or VMs, considering scalability, security, and application architecture.

2.6 Integrating VMs into Kubernetes with KubeVirt

2.6.1 KubeVirt Overview

Provide an in-depth introduction to KubeVirt, its objectives, architecture, and how it integrates VM management within Kubernetes.

2.6.2 Features and Capabilities

Detail the features KubeVirt offers, such as VM lifecycle management, network and storage integrations, and how it compares with traditional VM management approaches.

2.7 Evaluating VM Management Systems: Kubernetes with KubeVirt

2.7.1 Strategies for Evaluation

Outline methodologies for assessing VM management systems, including performance metrics, scalability tests, and operational efficiency.

2.7.2 Metrics for Comparison

Define specific metrics to be used in comparing KubeVirt against traditional VM management systems, such as deployment time, resource utilization, and ease of management.

2.8 Related Work

Chapter 3

Methods

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Chapter 4

Results

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Chapter 5

Discussion

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Chapter 6

Conclusions

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Bibliography

[1] Parth Yadav and Vipin Kumar Rathi. "KupenStack: Kubernetes based Cloud Native OpenStack". In: *arXiv preprint arXiv:2106.02956* (2021).

[2] Susanta Nanda Tzi-cker Chiueh and Stony Brook. "A survey on virtualization technologies". In: *Rpe Report* 142 (2005).

[3] Awodele Oludele et al. "On the evolution of virtualization and cloud computing: A review". In: *Journal of Computer Sciences and Applications* 2.3 (2014), pp. 40–43.

[4] M. Rosenblum and T. Garfinkel. "Virtual machine monitors: current technology and future trends". In: *Computer* 38.5 (2005), pp. 39–47. DOI: 10.1109/MC.2005.176.

[5] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Full and para-virtualization with Xen: a performance comparison". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.9 (2013), pp. 719–727.

[6] Tim Abels, Puneet Dhawan, and Balasubramanian Chandrasekaran. "An overview of xen virtualization". In: *Dell Power Solutions* 8 (2005), pp. 109–111.

[7] Fred Douglis and Orran Krieger. "Virtualization". In: *IEEE Internet Computing* 17.2 (2013), pp. 6–9. DOI: 10.1109/MIC.2013.42.

[8] Carl Boettiger. "An introduction to Docker for reproducible research". In: *SIGOPS Oper. Syst. Rev.* 49.1 (Jan. 2015). https://doi.org/10.1145/2723872.2723882, pp. 71–79. ISSN: 0163-5980. DOI: 10.1145/2723872.2723882.

[9] Lars Rasmusson and Diarmuid Corcoran. "Performance overhead of KVM on Linux 3.9 on ARM cortex-a15". In: *SIGBED Rev.* 11.2 (Sept. 2014). https://doi.org/10.1145/2668138.2668143, pp. 32–38. DOI: 10.1145/2668138.2668143.

[10] Ankita Desai et al. "Hypervisor: A survey on concepts and taxonomy". In: *International Journal of Innovative Technology and Exploring Engineering* 2.3 (2013), pp. 222–225.

[11] Stephen J. Bigelow. *What's the difference between type 1 vs. type 2 hypervisor?: TechTarget*. Jan. 2021. URL: https://www.techtarget.com/searchitoperations/tip/Whats-the-difference-between-Type-1-vs-Type-2-hypervisor.

[12] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.12 (2013), pp. 922–933.

[13] Jordan Shropshire. "Analysis of monolithic and microkernel architectures: Towards secure hypervisor design". In: *2014 47th Hawaii International Conference on System Sciences*. IEEE. 2014, pp. 5008–5017.

[14] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.12 (2013), pp. 922–933.

[15] Pankaj Kadam and Nilesh Alone. "Review on KVM Hypervisor". In: *International Journal of Recent Technology and Engineering (IJRTE) ISSN* (2014), pp. 2277–3878.

[16] Xiaolong Wen et al. "Comparison of open-source cloud management platforms: OpenStack and OpenNebula". In: *2012 9th International Conference on Fuzzy Systems and Knowledge Discovery*. IEEE. 2012, pp. 2457–2461.

[17] David Freet et al. "Open source cloud management platforms and hypervisor technologies: A review and comparison". In: *SoutheastCon 2016* (2016), pp. 1–8.

[18] Tiago Rosado and Jorge Bernardino. "An overview of openstack architecture". In: *Proceedings of the 18th International Database Engineering & Applications Symposium*. 2014, pp. 366–367.

[19] OpenStack Foundation. *OpenStack Live Migration*. Feb. 2024. URL: https://docs.openstack.org/nova/pike/admin/live-migration-usage.html.

[20] Rakesh Kumar et al. "Apache cloudstack: Open source infrastructure as a service cloud computing platform". In: *Proceedings of the International Journal of advancement in Engineering technology, Management and Applied Science* 111 (2014), p. 116.

[21] Apache CloudStack. *CloudStack Live Migration*. Feb. 2024. URL: https://svn.apache.org/repos/asf/cloudstack/docsite/html/docs/en-US/Apache_CloudStack/4.2.0/html/Admin_Guide/manual-live-migration.html.

[22] Daniel Moritz et al. "Enhancing a virtual security lab with a private cloud framework". In: *Proceedings of 2013 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)*. IEEE. 2013, pp. 314–320.

[23] Younes Khair, Abdeslam Dennai, and Youssef Elmir. "An experimental performance evaluation of OpenNebula and eucalyptus cloud platform solutions". In: *Artificial Intelligence and Heuristics for Smart Energy Efficiency in Smart Cities: Case Study: Tipasa, Algeria*. Springer. 2022, pp. 450–457.

[24] Anurag Choudhary, Pradeep Kumar Verma, and Piyush Rai. "A walkthrough of Amazon Elastic Compute Cloud (Amazon EC2): A Review". In: *International Journal for Research in Applied Science and Engineering Technology* 9.11 (2021), pp. 93–97.

[25] Amazon Web Services. *Amazon EC2*. Feb. 2024. URL: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html.

[26] Pallavi Wankhede, Minaiy Talati, and Rutuja Chinchamalatpure. "Comparative study of cloud platforms-microsoft azure, google cloud platform and amazon EC2". In: *J. Res. Eng. Appl. Sci* 5.02 (2020), pp. 60–64.

[27] Marshall Copeland et al. "Getting started with Azure virtual machines". In: *Microsoft Azure: Planning, Deploying, and Managing Your Data center in the Cloud* (2015), pp. 147–176.

[28] Rawan Aljamal, Fahed Jubair, and Ali El-Mousa. "Analyzing the Cost-Effectiveness of Running HPC Applications on Microsoft Azure Virtual Machines". In: *2021 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT)*. IEEE. 2021, pp. 78–82.

[29] Microsoft. *Azure Virtual Machines*. Feb. 2024. URL: https://learn.microsoft.com/en-us/azure/virtual-machines/overview.

[30] TechTarget. *Google Compute Engine*. Feb. 2024. URL: https://www.techtarget.com/searchaws/definition/Google-Compute-Engine#:~:text=Google%20Compute%20Engine%20(GCE)%20is,compute%20clusters%20for%20that%20purpose..

[31] SPT Krishnan et al. "Google compute engine". In: *Building Your Next Big Thing with Google Cloud Platform: A Guide for Developers and Enterprise Architects* (2015), pp. 53–81.

[32] Solanke Vikas et al. "Private vs public cloud". In: *International Journal of Computer Science & Communication Networks* 3.2 (2013), p. 79.

[33] Guruh Aryotejo, Daniel Y Kristiyanto, et al. "Hybrid cloud: bridging of private and public cloud computing". In: *Journal of Physics: Conference Series*. Vol. 1025. 1. IOP Publishing. 2018, p. 012091.

[34] Khouloud Boukadi et al. "From VM to container: A linear program for outsourcing a business process to cloud containers". In: *On the Move to Meaningful Internet Systems. OTM 2017 Conferences: Confederated International Conferences: CoopIS, C&TC, and ODBASE 2017, Rhodes, Greece, October 23-27, 2017, Proceedings, Part I*. Springer. 2017, pp. 488–504.

[35] Nane Kratzke. "About microservices, containers and their underestimated impact on network performance". In: *arXiv preprint arXiv:1710.04049* (2017).

[36] Arnaldo Pereira Ferreira and Richard Sinnott. "A performance evaluation of containers running on managed kubernetes services". In: *2019 IEEE international conference on cloud computing technology and science (CloudCom)*. IEEE. 2019, pp. 199–208.

[37] Vitor Goncalves da Silva, Marite Kirikova, and Gundars Alksnis. "Containers for Virtualization: An Overview." In: *Appl. Comput. Syst.* 23.1 (2018), pp. 21–27.

[38] Open Container Initiative. *Open Container Initiative*. Feb. 2024. URL: https://opencontainers.org/.

[39] Lennart Espe et al. "Performance Evaluation of Container Runtimes." In: *CLOSER*. 2020, pp. 273–281.

[40] Arun Gurta. “Kubernetes”. In: *Kubernetes. Retrieved May* 24 (2019), p. 2019.

[41] Kubernetes. *Custom Resource Definitions*. Feb. 2024. URL: https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/.

[42] Kakoli Banerjee et al. “A Survey on Kubernetes Policy Report Custom Resource Definition Kube-Bench Adapter”. In: *Advances in Data and Information Sciences: Proceedings of ICDIS 2022*. Springer, 2022, pp. 315–322.

[43] Prateek Sharma et al. “Containers and virtual machines at scale: A comparative study”. In: *Proceedings of the 17th international middleware conference*. 2016, pp. 1–13.

[44] Amazon Web Services. *Noisy Neighbor*. Feb. 2024. URL: https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/noisy-neighbor.html.

[45] Ishan Banerjee et al. “Memory overcommitment in the ESX server”. In: *VMware Technical Journal* 2.1 (2013), pp. 2–12.

[46] Jinchun Kim et al. “Dynamic memory pressure aware ballooning”. In: *Proceedings of the 2015 International Symposium on Memory Systems*. 2015, pp. 103–112.

[47] Christopher Clark et al. “Live migration of virtual machines”. In: *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2*. 2005, pp. 273–286.

[48] Radostin Stoyanov and Martin J Kollingbaum. “Efficient live migration of linux containers”. In: *High Performance Computing: ISC High Performance 2018 International Workshops, Frankfurt/Main, Germany, June 28, 2018, Revised Selected Papers 33*. Springer. 2018, pp. 184–193.

[49] CRIU. *CRIU*. Feb. 2024. URL: https://criu.org/.

Appendix A

Something Extra